

10:52

SIMATS O...  
veetha.comSIMATS Online Programming  
Lab

C++ PROGRAMMING

N K Jayasurya  
1924260172D

## CO5\_AT3\_CODE\_CHALLENGING

View Only

← Back

## QUESTIONS

Q1  
Deep Copy of Student Record  
20 marksQ2  
Virtual Destructor Debugging  
20 marksQ3  
Bank Account Runtime  
Polymorphism  
20 marksQ4  
Vehicle Management Using  
Virtual Functions  
20 marksQ5  
Multimedia Player Abstraction  
20 marks

5/5 answered

## Q1 Deep Copy of Student Record

20 marks

Debug a class containing dynamically allocated marks. Implement a deep-copy constructor and destructor. Changing one object's marks must not change the copied object.

```
class Student {  
    int* marks;  
public:  
    Student(int m) { marks = new int(m); }  
    Student(const Student& s) {  
        marks = s.marks; // Bug: shallow copy  
    }  
    void setMarks(int m) { *marks = m; }  
    void display() { cout << *marks << endl; }  
};
```

## Your Code

```
1 #include <iostream>  
2 using namespace std;  
3  
4 class Student {  
5     int* marks;  
6  
7 public:  
8     Student(int m) {  
9         marks = new int(m);  
10    }  
11  
12    // Deep Copy Constructor  
13    Student(const Student& s) {  
14        marks = new int(*s.marks);  
15    }  
16  
17    void setMarks(int m) {  
18        *marks = m;  
19    }  
20  
21    void display() {  
22        cout << *marks << endl;  
23    }  
24  
25    // Destructor  
26    ~Student() {  
27        delete marks;  
28    }  
29 };  
30  
31 int main() {  
32     Student s1(90);  
33  
34     Student s2 = s1;  
35  
36     s1.setMarks(100);  
37  
38     s1.display();  
39     s2.display();  
40  
41     return 0;  
42 }
```

## Input

stdin input

## Output



## Visible Test Cases

Test Case 1

10:54



SIMATS O...

veetha.com



## SIMATS Online Programming Lab

C++ PROGRAMMING

N K Jayasurya  
1924260172D

## CO5\_AT3\_CODE\_CHALLENGING

View Only

← Back

## QUESTIONS

Q1  
Deep Copy of Student Record  
20 marksQ2  
Virtual Destructor Debugging  
20 marksQ3  
Bank Account Runtime  
Polymorphism  
20 marksQ4  
Vehicle Management Using  
Virtual Functions  
20 marksQ5  
Multimedia Player Abstraction  
20 marks

5/5 answered

## Q2 Virtual Destructor Debugging

20 marks

Debug the following inheritance program. A derived object is deleted through a base pointer. Modify the base class so that both destructors execute safely and in the correct order.

```
class Parent {  
public:  
    ~Parent() { cout << "Parent Destructor" << endl; }  
};  
class Child : public Parent {  
public:  
    ~Child() { cout << "Child Destructor" << endl; }  
};  
int main() {  
    Parent* p = new Child();  
    delete p;  
}
```

## Your Code

```
1 #include <iostream>  
2 using namespace std;  
3  
4 class Parent {  
5 public:  
6     virtual ~Parent() {  
7         cout << "Parent Destructor" << endl;  
8     }  
9 };  
10  
11 class Child : public Parent {  
12 public:  
13     ~Child() {  
14         cout << "Child Destructor" << endl;  
15     }  
16 };  
17  
18 int main() {  
19     Parent* p = new Child();  
20  
21     delete p;  
22  
23     return 0;  
24 }
```

## Input

stdin input

## Output

## Visible Test Cases

Test Case 1

10:55



SIMATS O...

veetha.com



## SIMATS Online Programming Lab

C++ PROGRAMMING

N K Jayasurya  
1924260172D

## CO5\_AT3\_CODE\_CHALLENGING

View Only

← Back

## QUESTIONS

Q1  
Deep Copy of Student Record  
20 marksQ2  
Virtual Destructor Debugging  
20 marksQ3  
Bank Account Runtime Polymorphism  
20 marksQ4  
Vehicle Management Using Virtual Functions  
20 marksQ5  
Multimedia Player Abstraction  
20 marks

5/5 answered

## Q3 Bank Account Runtime Polymorphism

20 marks

Write a C++ program using an abstract Account class and derived SavingsAccount, CurrentAccount, and LoanAccount classes. Implement calculateInterest() using runtime polymorphism. Identify and correct the starter-code error so a base pointer invokes the derived implementation.

## Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Account {
5 public:
6     virtual void calculateInterest() = 0;
7     virtual ~Account() {}
8 };
9
10
11 class SavingsAccount : public Account {
12     double balance;
13 public:
14     SavingsAccount(double b) {
15         balance = b;
16     }
17     void calculateInterest() override {
18         cout << "Savings Interest: " << balance * 0.05 << endl;
19     }
20 };
21
22
23 class CurrentAccount : public Account {
24     double balance;
25 public:
26     CurrentAccount(double b) {
27         balance = b;
28     }
29     void calculateInterest() override {
30         cout << "Current Interest: " << balance * 0.03 << endl;
31     }
32 };
33
34
35 class LoanAccount : public Account {
36     double amount;
37 public:
38     LoanAccount(double a) {
39         amount = a;
40     }
41     void calculateInterest() override {
42         cout << "Loan Interest: " << amount * 0.10 << endl;
43     }
44 };
45
46
47 int main() {
48     double amount;
49     cin >> amount;
50     Account* account;
51     SavingsAccount savings(amount);
52     CurrentAccount current(amount);
53     LoanAccount loan(amount);
54     account = &savings;
55     account->calculateInterest();
56     account = &current;
57     account->calculateInterest();
58     account = &loan;
59     account->calculateInterest();
60     return 0;
61 }
```

## Input

10000

## Output

## Visible Test Cases

Test Case 1

10:56

SIMATS O...  
veetha.comSIMATS Online Programming  
Lab  
C++ PROGRAMMINGN K Jayasurya  
1924260172D

## CO5\_AT3\_CODE\_CHALLENGING

View Only

← Back

## QUESTIONS

Q1  
Deep Copy of Student Record  
20 marksQ2  
Virtual Destructor Debugging  
20 marksQ3  
Bank Account Runtime  
Polymorphism  
20 marksQ4  
Vehicle Management Using  
Virtual Functions  
20 marksQ5  
Multimedia Player Abstraction  
20 marks

5/5 answered

## Q4 Vehicle Management Using Virtual Functions

20 marks

Create a Vehicle base class with virtual displayDetails(). Implement Car, Bike and Bus. Store derived objects through Vehicle pointers and display the correct information without type-based if/else statements.

## Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Vehicle {
5 public:
6     virtual void displayDetails() {
7         cout << "Vehicle" << endl;
8     }
9
10    virtual ~Vehicle() {}
11 };
12
13 class Car : public Vehicle {
14 public:
15     void displayDetails() override {
16         cout << "Car" << endl;
17     }
18 };
19
20 class Bike : public Vehicle {
21 public:
22     void displayDetails() override {
23         cout << "Bike" << endl;
24     }
25 };
26
27 class Bus : public Vehicle {
28 public:
29     void displayDetails() override {
30         cout << "Bus" << endl;
31     }
32 };
33
34 int main() {
35     Vehicle* vehicles[3];
36
37     Car car;
38     Bike bike;
39     Bus bus;
40
41     vehicles[0] = &car;
42     vehicles[1] = &bike;
43     vehicles[2] = &bus;
44
45     for (int i = 0; i < 3; i++) {
46         vehicles[i]->displayDetails();
47     }
48
49     return 0;
50 }
```

## Input

stdin input

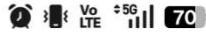
## Output

## Visible Test Cases

Test Case 1



10:57

SIMATS O...  
veetha.comSIMATS Online Programming  
Lab

C++ PROGRAMMING

N K Jayasurya  
19242831720

C05\_AT3\_CODE\_CHALLENGING

View Only

← Back

## QUESTIONS

Q1

Deep Copy of Student Record  
20 marks

Q2

Virtual Destructor Debugging  
20 marks

Q3

Bank Account Runtime  
Polymorphism  
20 marks

Q4

Vehicle Management Using  
Virtual Functions  
20 marks

Q5

Multimedia Player Abstraction  
20 marks

5/5 answered

## Q5 Multimedia Player Abstraction

20 marks

Create an abstract MediaPlayer class with a pure virtual play(). Implement MP3Player, WAVPlayer, MP4Player, and AVIPlayer. Select the correct object from the input and invoke play() polymorphically.

## Your Code

```
#include <iostream>
using namespace std;

class MediaPlayer {
public:
    virtual void play() = 0;
    virtual ~MediaPlayer() {}
};

class MP3Player : public MediaPlayer {
public:
    void play() override {
        cout << "Playing MP3" << endl;
    }
};

class WAVPlayer : public MediaPlayer {
public:
    void play() override {
        cout << "Playing WAV" << endl;
    }
};

class MP4Player : public MediaPlayer {
public:
    void play() override {
        cout << "Playing MP4" << endl;
    }
};

class AVIPlayer : public MediaPlayer {
public:
    void play() override {
        cout << "Playing AVI" << endl;
    }
};

int main() {
    int choice;
    cin >> choice;

    MediaPlayer* player;

    MP3Player mp3;
    WAVPlayer wav;
    MP4Player mp4;
    AVIPlayer avi;

    if (choice == 1)
        player = &mp3;
    else if (choice == 2)
        player = &wav;
    else if (choice == 3)
        player = &mp4;
    else
        player = &avi;

    player->play();

    return 0;
}
```

## Input

1

## Output

## Visible Test Cases

Test Case 1